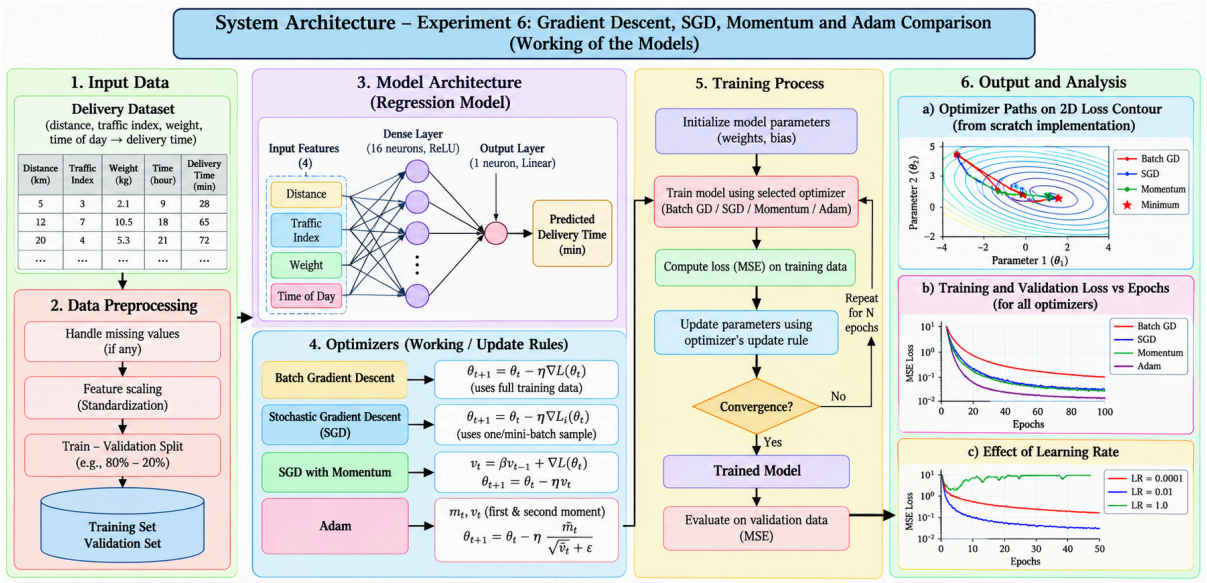
Experiment 6: Gradient Descent, SGD, Momentum and Adam Comparison

1. AIM

To implement and compare Batch Gradient Descent (GD), Stochastic Gradient Descent (SGD), SGD with Momentum, and Adam optimizer for a delivery-time prediction model. To visualize optimizer paths on a two-dimensional loss contour, compare loss against epochs, and study the effects of different learning rates on convergence using NumPy, TensorFlow, Keras, and Matplotlib.



2. MATHEMATICAL FORMULAS

1. Mean Squared Error (MSE)

$$\boxed{L(\theta) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

2. Batch Gradient Descent

$$\boxed{\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t)}$$

3. Stochastic Gradient Descent (SGD)

$$\boxed{\theta_{t+1} = \theta_t - \eta \nabla_\theta L_i(\theta_t)}$$

4. SGD with Momentum

$$\boxed{v_t = \beta v_{t-1} + \nabla_\theta L(\theta_t)}$$

$$\boxed{\theta_{t+1} = \theta_t - \eta v_t}$$

5. Adam Optimizer

First moment estimate:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

Second moment estimate:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Bias correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Parameter update:

$$\boxed{\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}}$$

6. Learning Rate

CODE

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error

np.random.seed(42)
tf.random.set_seed(42)

# ========================================================
# PART 1: IMPLEMENT OPTIMIZERS FROM SCRATCH
# ========================================================

# Simple two-parameter loss function
def loss_function(theta):
    x, y = theta
    return (x - 2)**2 + 4 * (y - 1)**2

def gradient(theta):
    x, y = theta
    return np.array([2 * (x - 2), 8 * (y - 1)])

def optimize(method, lr=0.1, steps=100, beta=0.9):
    theta = np.array([-3.0, 4.0])
    velocity = np.zeros(2)
    path = [theta.copy()]

    for t in range(steps):
        if method == "Batch GD":
            g = gradient(theta)

        elif method == "SGD":
            # Noisy gradient simulates a randomly selected sample
            g = gradient(theta) + np.random.normal(0, 0.5, 2)

        elif method == "Momentum":
            g = gradient(theta)
            velocity = beta * velocity + g
            theta = theta - lr * velocity
            path.append(theta.copy())
            continue

        theta = theta - lr * g
        path.append(theta.copy())

    return np.array(path)

# Plot optimizer paths on the loss contour
x_grid = np.linspace(-4, 4, 150)
y_grid = np.linspace(-2, 5, 150)
XX, YY = np.meshgrid(x_grid, y_grid)
ZZ = (XX - 2)**2 + 4 * (YY - 1)**2
```

```
plt.figure(figsize=(9, 6))
plt.contour(XX, YY, ZZ, levels=25)

for method in ["Batch GD", "SGD", "Momentum"]:
    path = optimize(method, lr=0.04, steps=60)
    plt.plot(path[:, 0], path[:, 1],
             marker=".", markersize=3, label=method)

plt.scatter(2, 1, marker="*", s=180, label="Minimum")
plt.xlabel("Parameter 1")
plt.ylabel("Parameter 2")
plt.title("Optimizer Paths on a 2D Loss Contour")
plt.legend()
plt.grid(alpha=0.2)
plt.show()

# ======================================================
# PART 2: CREATE DELIVERY-TIME DATASET
# ======================================================

n = 500

distance = np.random.uniform(1, 30, n)
traffic = np.random.uniform(1, 10, n)
weight = np.random.uniform(0.5, 20, n)
time_of_day = np.random.uniform(0, 24, n)

# Illustrative delivery time in minutes
delivery_time = (
    10
    + 1.8 * distance
    + 2.5 * traffic
    + 0.4 * weight
    + 0.3 * np.abs(time_of_day - 13)
    + np.random.normal(0, 3, n)
)

X = np.column_stack([
    distance, traffic, weight, time_of_day
])
y = delivery_time.reshape(-1, 1)

X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Standardize inputs and target using training data only
x_scaler = StandardScaler()
y_scaler = StandardScaler()

X_train = x_scaler.fit_transform(X_train)
X_val = x_scaler.transform(X_val)

y_train = y_scaler.fit_transform(y_train)
y_val = y_scaler.transform(y_val)

# ======================================================
# PART 3: COMPARE KERAS OPTIMIZERS
```

```
# ====================================================

optimizer_factories = {
    "Batch GD": lambda: tf.keras.optimizers.SGD(
        learning_rate=0.01
    ),
    "SGD": lambda: tf.keras.optimizers.SGD(
        learning_rate=0.01
    ),
    "Momentum": lambda: tf.keras.optimizers.SGD(
        learning_rate=0.01, momentum=0.9
    ),
    "Adam": lambda: tf.keras.optimizers.Adam(
        learning_rate=0.01
    )
}

histories = {}
models = {}
results = []

for name, optimizer_factory in optimizer_factories.items():
    tf.keras.backend.clear_session()

    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(4,)),
        tf.keras.layers.Dense(16, activation="relu"),
        tf.keras.layers.Dense(1)
    ])

    model.compile(
        optimizer=optimizer_factory(),
        loss="mse"
    )

    # Batch GD uses the full training set per update.
    # SGD, Momentum and Adam use mini-batches.
    batch_size = len(X_train) if name == "Batch GD" else 32

    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=100,
        batch_size=batch_size,
        verbose=0
    )

    histories[name] = history.history
    models[name] = model

    pred_scaled = model.predict(X_val, verbose=0)
    pred_minutes = y_scaler.inverse_transform(pred_scaled)
    actual_minutes = y_scaler.inverse_transform(y_val)

    val_mse = mean_squared_error(actual_minutes, pred_minutes)

    results.append({
        "Optimizer": name,
```

```
        "Final Training Loss": history.history["loss"][-1],
        "Final Validation Loss": history.history["val_loss"][-1],
        "Validation MSE (minutes squared)": val_mse
    })

# Plot loss against epochs
plt.figure(figsize=(10, 6))

for name, history in histories.items():
    plt.plot(history["loss"], label=name)

plt.xlabel("Epoch")
plt.ylabel("Training MSE Loss")
plt.title("Training Loss vs Epochs")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

# Plot validation loss against epochs
plt.figure(figsize=(10, 6))

for name, history in histories.items():
    plt.plot(history["val_loss"], label=name)

plt.xlabel("Epoch")
plt.ylabel("Validation MSE Loss")
plt.title("Validation Loss vs Epochs")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

# Display comparison table
import pandas as pd

results_df = pd.DataFrame(results)
print("Optimizer Comparison:")
print(results_df.round(4).to_string(index=False))

# ====================================================
# PART 4: LEARNING RATE EXPERIMENT
# ====================================================

learning_rates = [0.0001, 0.01, 1.0]
lr_results = {}

for lr in learning_rates:
    tf.keras.backend.clear_session()

    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(4,)),
        tf.keras.layers.Dense(16, activation="relu"),
        tf.keras.layers.Dense(1)
    ])

    model.compile(
        optimizer=tf.keras.optimizers.SGD(
            learning_rate=lr
        ),
```

```
        loss="mse"
    )

    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=50,
        batch_size=32,
        verbose=0
    )

    lr_results[lr] = history.history["loss"]

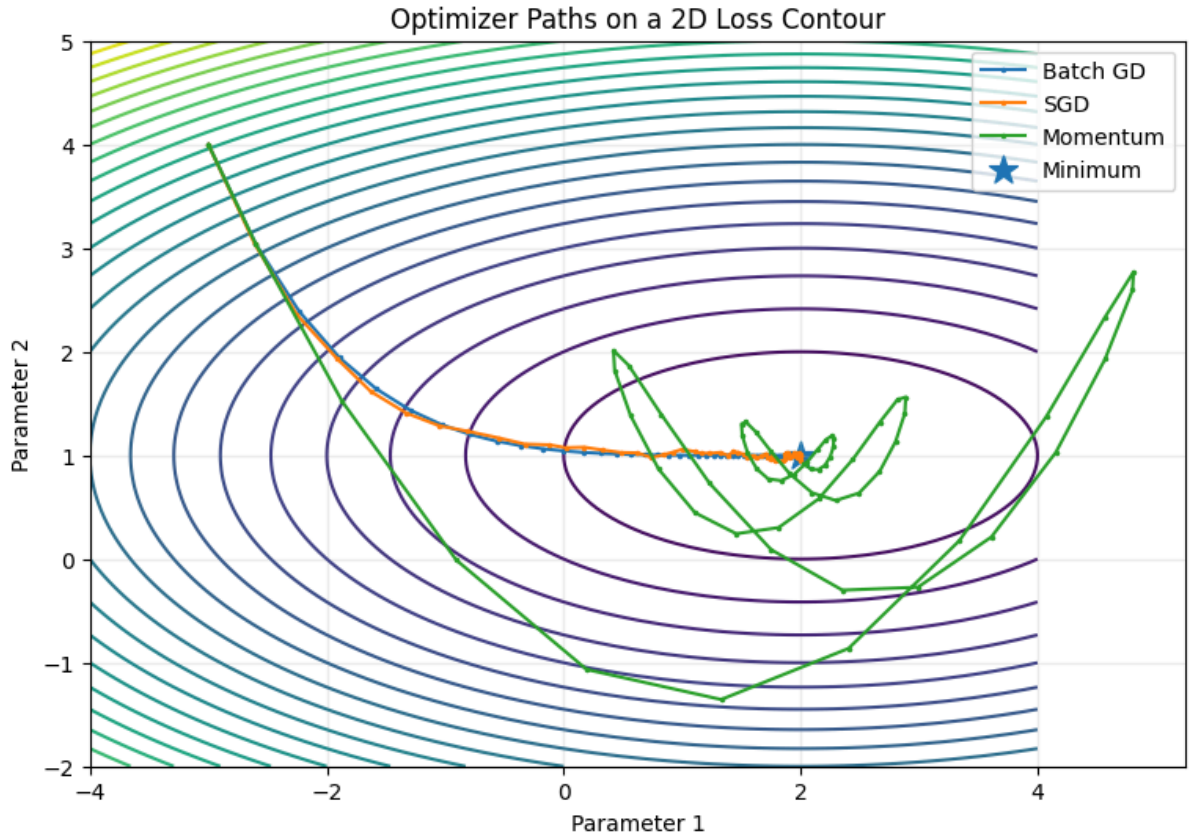
plt.figure(figsize=(9, 5))

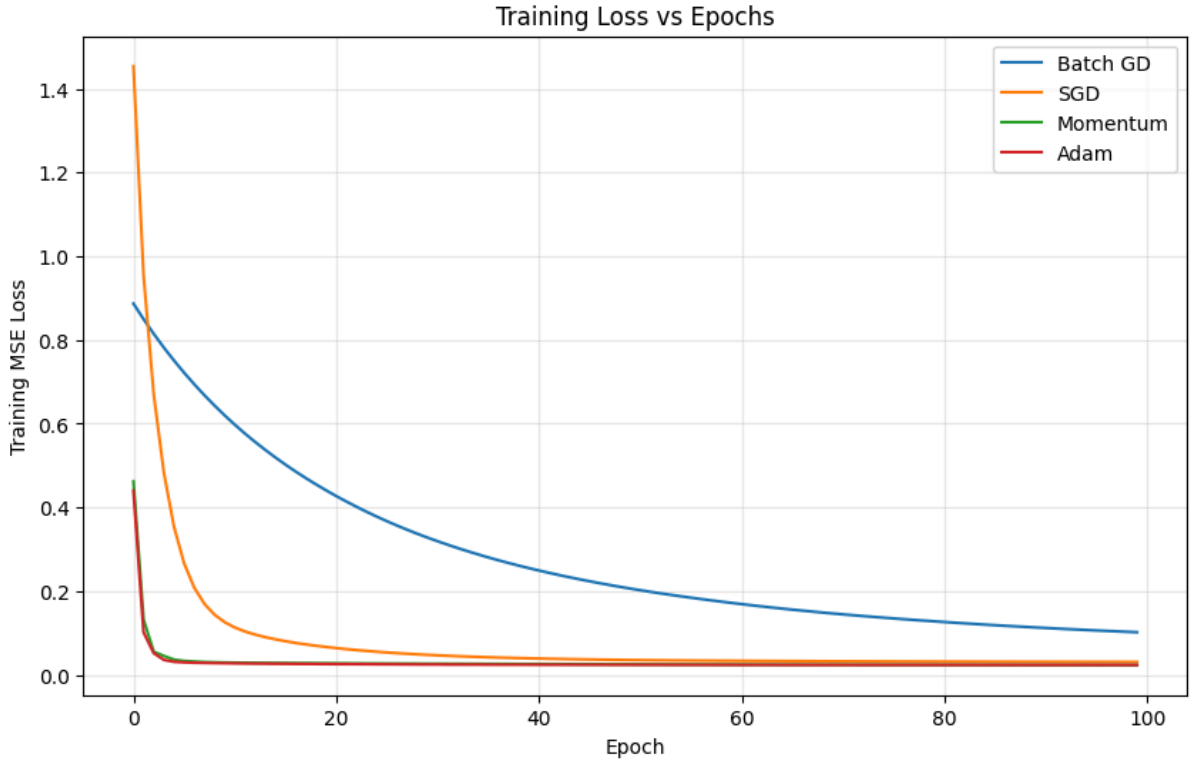
for lr, losses in lr_results.items():
    plt.plot(losses, label=f"Learning rate = {lr}")

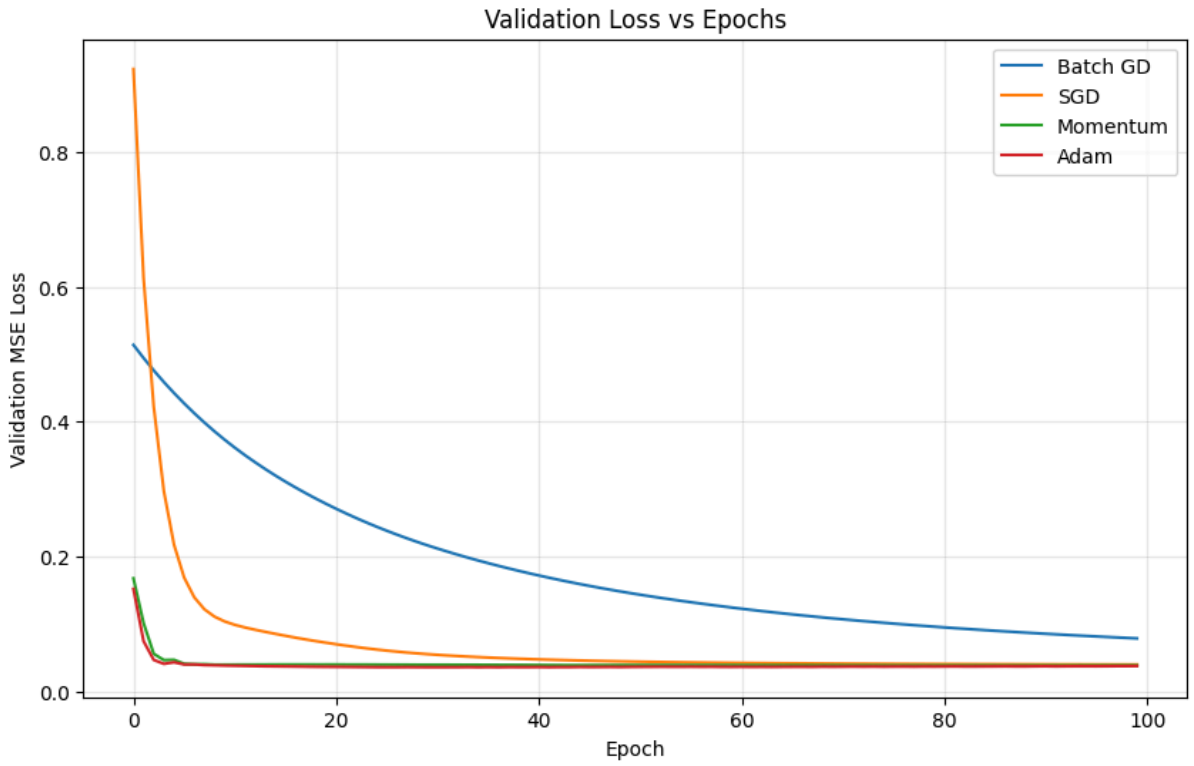
plt.xlabel("Epoch")
plt.ylabel("Training MSE Loss")
plt.title("Effect of Learning Rate on Convergence")
plt.yscale("log")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

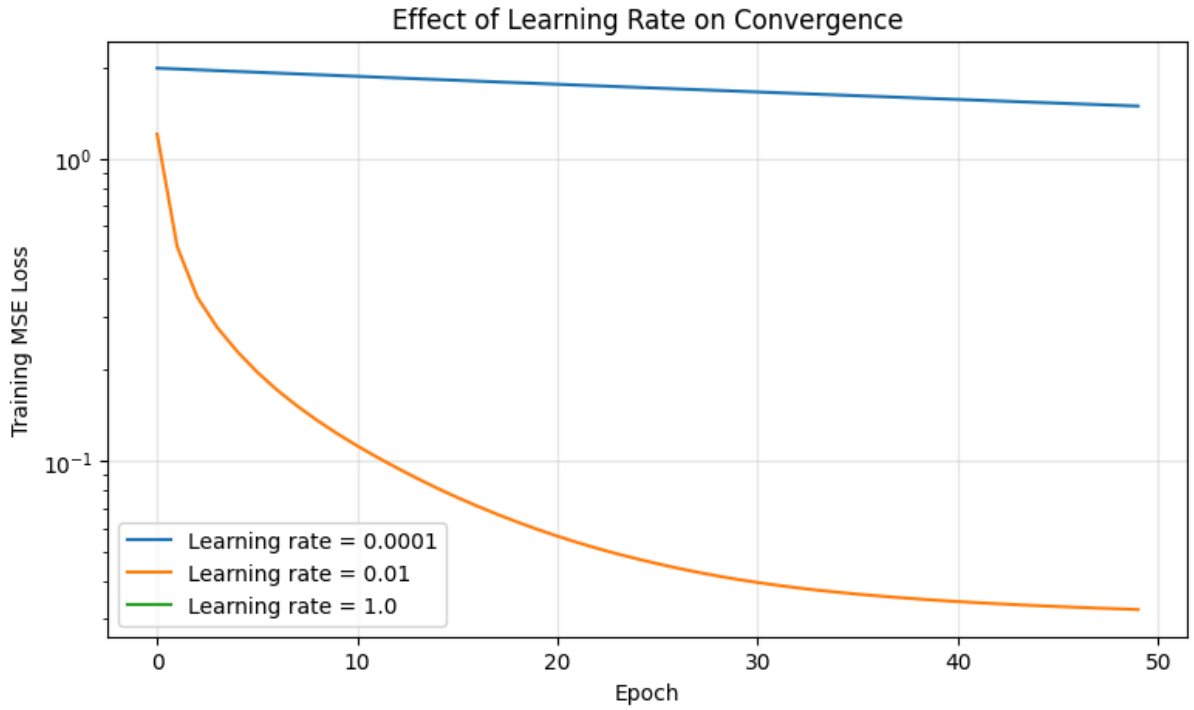
print("\nLearning Rate Analysis:")
for lr, losses in lr_results.items():
    print(
        f"Learning rate {lr}: "
        f"Initial loss = {losses[0]:.4f}, "
        f"Final loss = {losses[-1]:.4f}"
    )
```

OUTPUT


Optimizer Paths on a 2D Loss Contour
Batch GD
SGD
Momentum
Minimum
Parameter 2
Parameter 1
5
4
3
2
1
0
−1
−2
−4
−2
0
2
4


Training Loss vs Epochs
Batch GD
SGD
Momentum
Adam
Training MSE Loss
Epoch
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
0
20
40
60
80
100






```
Learning Rate Analysis:
Learning rate 0.0001: Initial loss = 1.9896, Final loss = 1.4909
Learning rate 0.01: Initial loss = 1.2033, Final loss = 0.0323
Learning rate 1.0: Initial loss = nan, Final loss = nan
```

5. RESULT

Batch Gradient Descent, Stochastic Gradient Descent, Momentum, and Adam were implemented and compared for delivery-time prediction. The optimizer paths were visualized on a two-dimensional loss contour, and training and validation loss curves were plotted. The effect of learning rate on convergence was also investigated. The experiment demonstrates that optimizer performance depends on the update strategy, learning rate, and data. The most suitable optimizer should be selected based on stable convergence and low validation error.